Consistency Models in Distributed Systems

What your database actually promises — and what it doesn't

A first-principles walk through every consistency model, from linearizability to eventual consistency, and how to choose.

Samuel Thien

THE PROBLEM

The Checkout That Broke

You're running an online store. A customer places an order. Behind the scenes, five things happen:

1. **Reserve inventory** (database write)
2. **Create order record** (database write)
3. **Charge payment** (Stripe API call)
4. **Notify warehouse** (message queue)
5. **Send confirmation email** (async job)

The server crashes after step 3. The customer is charged. The warehouse never got the message. Now multiply this by three replicas across two regions. **Which replica has the truth?**

When nodes disagree about state, "who is right?" is the wrong question.

The right question is: ***what are the rules?***

The rules are called **consistency models** — contracts between a system and its clients about which observations are legal.

Foundations

FOUNDATIONS

What Is a Consistency Model?

A **consistency model** defines which histories a system may legally produce.

- A **history** is a sequence of operations (reads, writes) across processes
- Some histories make intuitive sense. Others are bizarre.
- A consistency model draws the line: these histories are legal, those are not.

Stronger model = fewer legal histories = more predictable, more expensive
Weaker model = more legal histories = less predictable, cheaper

LINEARIZABILITY

The Strongest Single-Object Model: Linearizability

Herlihy & Wing, 1990:

> *Every operation appears to take effect **atomically** at some point between its **invocation** and its **response**.*

Three constraints:

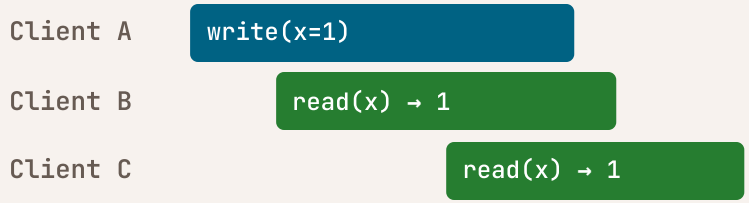
1. **Atomicity**: each operation happens at a single instant
2. **Real-time order**: if op A completes before op B starts, A must appear first
3. **Legal sequential spec**: the resulting sequence must be valid for the object

Linearizability = single copy + atomic updates + respect real-time order.

LINEARIZABILITY

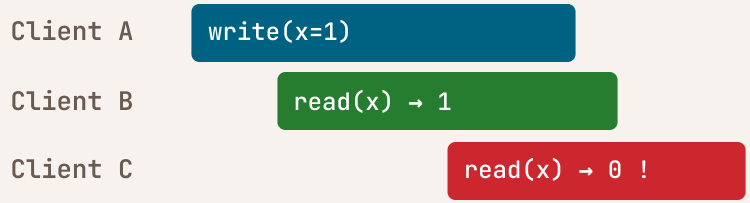
Linearizability: Visualized

Linearizable ✓



B and C both see 1 after A's write takes effect.

NOT Linearizable ×



C reads 0 **after** B already observed 1. No valid linearization.

Once a value is observed by any client, it cannot un-happen for any other client.

"The system behaves like a single copy, updated atomically."

— Herlihy & Wing, 1990

SEQUENTIAL CONSISTENCY

What If We Drop Real-Time Order?

Sequential consistency (Lamport, 1979):

All operations appear to execute in **some** sequential order, and each process's operations appear in **program order**.

But: two concurrent operations can be reordered relative to real time.

Property	Linearizable	Sequentially consistent
Per-process order preserved	Yes	Yes
Real-time order preserved	Yes	**No**
Appears atomic	Yes	Yes

Sequential consistency is linearizability minus the real-time constraint. Cheaper, but allows "time travel" between processes.

CAUSAL CONSISTENCY

What If We Only Preserve Causality?

If operation A *could have influenced* operation B, then everyone sees A before B.

But: **concurrent operations** (no causal relationship) can be seen in different orders by different processes.

Three rules define "could have influenced":

1. **Same process**: A before B in the same process → A causally precedes B
2. **Reads-from**: B reads a value written by A → A causally precedes B
3. **Transitivity**: A → B and B → C means A → C

EVENTUAL CONSISTENCY

What If We Guarantee... Almost Nothing?

Eventual consistency: if no new writes are made, all replicas will *eventually* converge to the same state.

What this does **not** specify:

- **When** convergence happens (no time bound)
- **What** intermediate values clients may observe
- **Which** value wins if there are conflicts

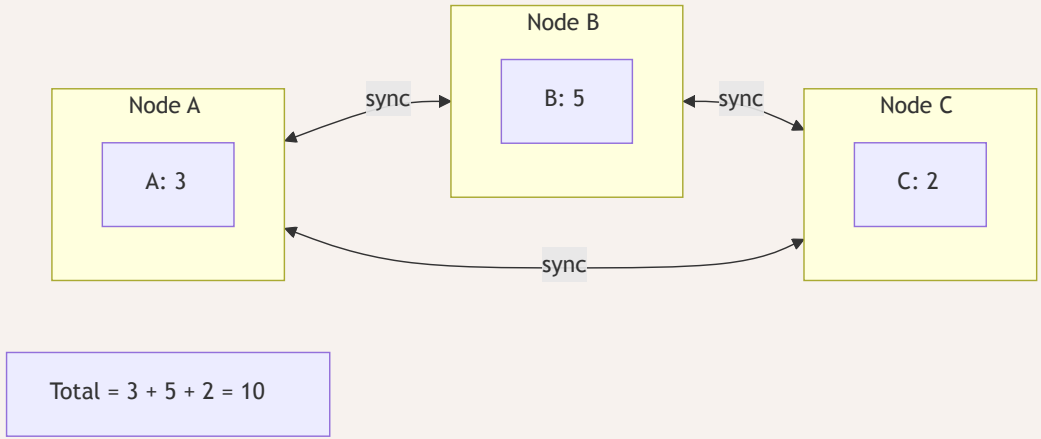
"Eventually consistent" is **not a guarantee**. It is the **absence** of one.

EVENTUAL CONSISTENCY

CRDTs: Convergence by Construction

CRDTs (Conflict-free Replicated Data Types) are data structures designed so all concurrent operations commute. Convergence is **automatic** and **deterministic**: no conflict resolution needed.

G-Counter: each node maintains its own counter slot. The total is always the sum of all slots. Increments never conflict because nodes only write to their own slot.



Other examples: OR-Set (add/remove sets), LWW-Register (last-writer-wins per cell)

EVENTUAL CONSISTENCY

Bounded Staleness: Putting a Clock on "Eventually"

Eventual consistency becomes a real guarantee when you bound the lag:

Bounded staleness: "Replicas lag by at most *k* versions or *t* seconds"

- Spanner offers bounded-staleness reads as a cheaper alternative to linearizable reads
- DynamoDB's DAX cache provides bounded-stale reads within milliseconds
- This turns an open-ended promise into a measurable SLA

The real spectrum: not "strong vs eventual" but linearizable → sequential → causal → bounded-stale → eventual

The real spectrum is not "strong vs eventual." It's a rich middle ground: linearizable → sequential → causal → bounded-stale → eventual. CRDTs eliminate conflicts; bounded staleness limits how far behind you can fall. Together, they make eventual consistency usable.

So far: one object, many observers.

But what about operations that touch **multiple objects**?

Two Dimensions of Consistency

SERIALIZABILITY

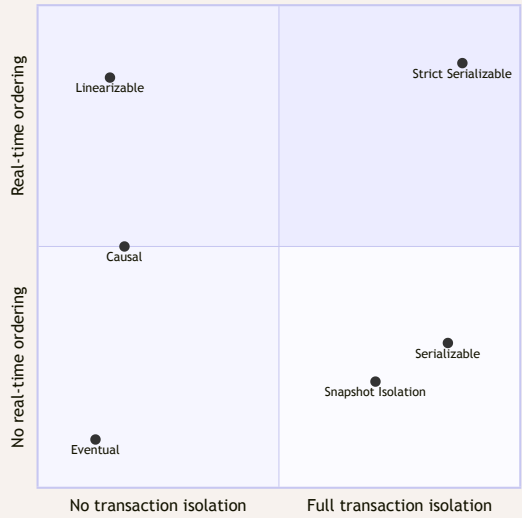
Single-Object vs Multi-Object Consistency

Single-object consistency(one key/register)

- Linearizability
- Sequential consistency
- Causal consistency
- Eventual consistency

Multi-object consistency(transactions across keys)

- Serializability
- Snapshot isolation
- Read committed
- Read uncommitted



These are **independent dimensions** that can be combined, not alternatives on the same axis. Most talks and most engineers conflate them.

SERIALIZABILITY

Serializability: Transactions in Some Order

The result of executing transactions concurrently is equivalent to executing them in **some** sequential order.

Key distinction from linearizability:

- No real-time constraint — the "some order" can differ from wall-clock order
- Applies to **transactions** (groups of operations), not individual operations
- A transaction that reads A and writes B is atomic

Serializability guarantees a sequential explanation exists. It doesn't promise **which** sequence, or that it matches wall-clock time.

SNAPSHOT ISOLATION

Snapshot Isolation: Almost Serializable

Snapshot isolation (Berenson et al., 1995):

1. Each transaction reads from a **consistent snapshot** taken at its start
2. Writes are buffered and applied atomically at commit
3. **First-committer-wins**: if two transactions write the same key, the second to commit is aborted

What SI **prevents**: dirty reads, non-repeatable reads, phantom reads, **lost updates**

What SI **allows**: **write skew** — and this is why SI is not serializable

SNAPSHOT ISOLATION

The Write Skew Problem

Setup: Hospital requires at least one doctor on call. Alice and Bob are both on call.

STRICT SERIALIZABILITY

The Strongest Model: Strict Serializability

Strict serializability = Serializability + Linearizability

- Transactions execute as if in some sequential order (**serializability**)
- AND that order must respect real-time precedence (**linearizability**)

If transaction A commits before transaction B starts, A appears before B in the serial order. No exceptions.

This is what Google Spanner calls **external consistency**.

Model	Single-object real-time	Multi-object serial order	Both
Linearizable	Yes	—	—
Serializable	—	Yes	—
Strict serializable	**Yes**	**Yes**	**Yes**

Linearizable is about objects.
Serializable is about transactions.
Strict serializable is about both.

The Session Layer

SESSION GUARANTEES

Three Layers of Consistency

Layer 1: System-level — what the database guarantees globally "All reads reflect the latest committed write" (linearizable)"All replicas converge" (eventually consistent)

Layer 2: Session-level — what a single client connection sees "You always see your own writes" (read-your-writes)"Values never go backward" (monotonic reads)

Layer 3: Application-level — what the developer must enforce "The sum of debits equals the sum of credits" (invariant)"Only one winner per auction" (constraint)

A system with weak system-level consistency can still provide strong session guarantees. **These layers are independent.**

SESSION GUARANTEES

Session Guarantees (Terry et al., 1994)

Four session-level guarantees, each independent:

Read Your Writes: a read following a write in the same session always reflects that write (or a later one)

Monotonic Reads: successive reads in a session never return older values — time doesn't go backward

Monotonic Writes: writes from a session are applied in the order they were issued

Writes Follow Reads: a write that follows a read in a session is ordered after the value that was read

Cheap to implement (sticky sessions, version vectors) and dramatically improve developer experience on eventually-consistent systems.

The system guarantees X.

The session guarantees Y.

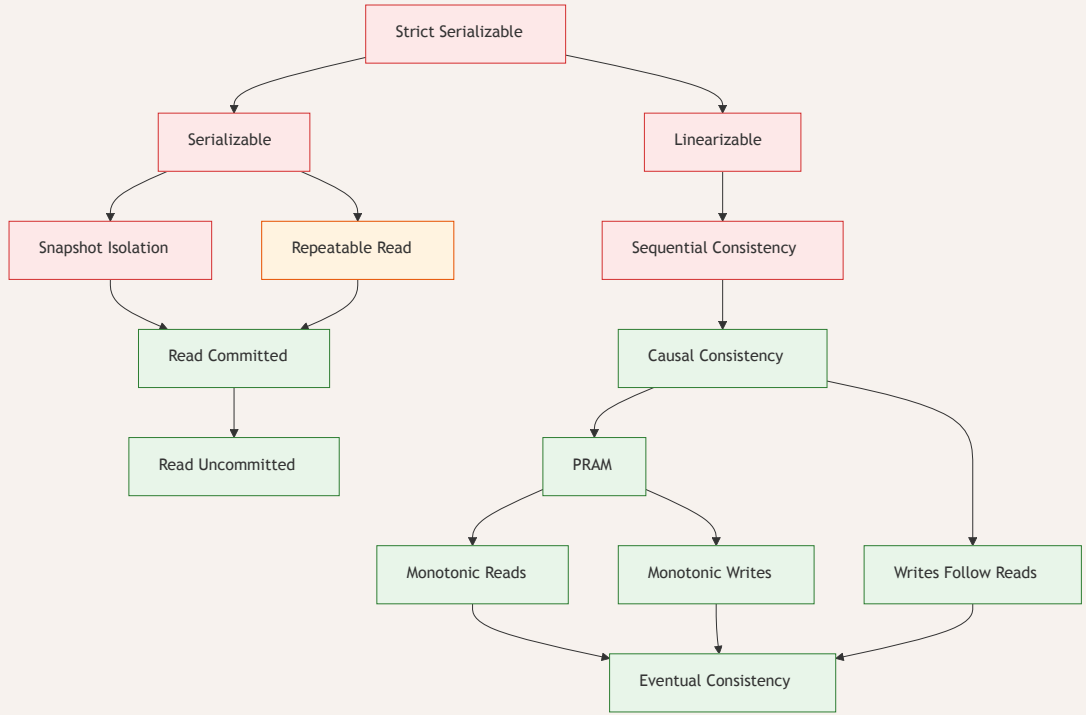
The developer enforces Z.

The Hierarchy

THE HIERARCHY

The Consistency Hierarchy

Adapted from Jepsen / Bailis, Davidson, Fekete et al. — arrows show implication (higher implies lower).



Unavailable under partition Depends on implementation Available under partition

Every consistency model is defined by the constraint it drops.

Drop real-time order → **sequential consistency**

Drop total order → **causal consistency**

Drop causal order → **eventual consistency**

Drop cross-key atomicity → **snapshot isolation**

Drop real-time + cross-key → **most production databases**

THE HIERARCHY

What This Hierarchy Does NOT Mean

"Linearizable" does NOT mean "serializable" They are independent dimensions. Linearizability is about one object in real time. Serializability is about transactions across objects. You can have either without the other.

"CAP" does NOT mean "pick 2 of 3" Partitions are not optional. CAP says: *during* a partition, choose availability or consistency. Outside partitions, you make a different trade-off (latency vs. consistency).

"Eventually consistent" does NOT mean "inconsistent" It means the timing of convergence is unspecified. With CRDTs or bounded staleness, eventual consistency can be a precise, useful guarantee.

Real Systems, Real Choices

Every consistency choice was forced by a specific production failure.

Not academic preference. Operational necessity.

AMAZON DYNAMO

Amazon Dynamo (2007): Availability Over Everything

The problem: Amazon's peak shopping season. Every 100ms of added latency cost 1% of sales. A database that rejected writes during a network partition meant customers couldn't add to cart.

The choice: Eventual consistency with client-side conflict resolution.

The mechanism:

- Sloppy quorum: writes are accepted by any N healthy nodes, even if they're not the designated replicas for that key. Writes always succeed, even during partition.
- Vector clocks (per-key version vectors) track causal history across replicas
- Conflicting versions pushed to application layer ("add to cart" merges via union)

The trade-off: Developers must handle conflicts. Shopping cart: easy (union of items). Payment processing: impossible (you can't "merge" two charges).

DeCandia et al., SOSP 2007

GOOGLE SPANNER

Google Spanner (2012): The Price of Weak Consistency at Scale

The problem: Google Ads ran on sharded MySQL. Cross-shard transactions required manual two-phase commit at the application layer. Weaker consistency meant ad budgets could overspend or ads could serve after campaigns ended.

The choice: External consistency (= strict serializability) via TrueTime.

The mechanism:

- Atomic clocks + GPS receivers in every data center
- TrueTime API returns an interval [earliest, latest] instead of a single timestamp
- Commit wait: delay commits by the uncertainty interval (~7ms average)
- Paxos (consensus protocol for keeping replicas in sync) replication runs during the wait, so it's not wasted time

The trade-off: ~7ms commit latency floor. Custom hardware. Acceptable for Ads (billions in revenue depend on correctness).

Corbett et al., OSDI 2012

COCKROACHDB

CockroachDB: Serializable by Default

The problem: PostgreSQL defaults to Read Committed. Most applications assume serializable behavior. Write skew bugs are silent and rare enough to escape testing — but catastrophic when they hit production.

The choice: Serializable isolation as the default. Opt down, don't opt up.

	PostgreSQL	CockroachDB
Default isolation	Read Committed	**Serializable**
Write skew prevention	Only at SERIALIZABLE	**Always**
Transaction retry	Rare	More frequent
Developer burden	Must know to opt up	Must know to opt down

The philosophy: Correct by default. Performance tuning is opt-in.

REAL SYSTEMS

Write Skew in SQL: What It Actually Looks Like

PostgreSQL (Read Committed) — breaks silently

```
-- Both sessions run concurrently:
BEGIN;
SELECT count(*) FROM doctors
  WHERE on_call = true; -- → 2
UPDATE doctors SET on_call = false
  WHERE name = 'Alice'; -- (or Bob)
COMMIT; -- Both succeed
-- Result: 0 doctors on call!
```

CockroachDB (Serializable) — caught automatically

```
-- Same SQL, same timing:
ERROR: restart transaction
  REASON: WriteTooOld

-- One transaction retried.
-- Invariant preserved:
-- 1 doctor remains on call.
```

Same application code, different database defaults, opposite outcomes. The isolation level is not an optimization knob — it's a correctness knob.

CAP VS PACELC

PACELC: The Trade-off That Actually Matters

CAP (Brewer, 2000; Gilbert & Lynch, 2002): During a **network partition**, choose **availability** or **consistency**.

What CAP does **not** say:

- It does NOT apply during normal operation
- It is NOT "pick 2 of 3" — you always get P (partitions happen)
- It does NOT address latency at all

PACELC (Abadi, 2012): If **P**artition → choose **A** or **C**. Else → choose **L**atency or **C**onsistency.

CAP is about once-a-year disasters.
PACELC is about every request.

Failure Modes in Production

Theory tells you what can go wrong.

Production tells you what **will** go wrong.

FAILURE MODES

Stale Reads After Leader Failover

A Raft-based system (etcd, Consul) claims linearizability. A network partition isolates the leader.

FAILURE MODES

Read Skew vs Write Skew

Read Skew

Transaction reads multiple related values at different points in time, seeing an inconsistent combination.

```
T1: READ balance_a → $500
    (T2 transfers $100: a→b)
T1: READ balance_b → $600
T1 sees total = $1100
Actual total = $1000
```

Prevented by: **Repeatable Read** and above

Write Skew

Two transactions read shared state, then write to disjoint keys, together violating an invariant.

```
T1: READ doctors_on_call → 2
T2: READ doctors_on_call → 2
T1: WRITE alice = off_call
T2: WRITE bob = off_call
Result: 0 doctors on call
```

Prevented by: **Serializable** only

Read skew is a stale snapshot. Write skew is a logical race condition. Different anomalies, different cures.

FAILURE MODES

The Cost of Linearizability

Linearizable operations must reflect **global state**. Every linearizable read must verify it's seeing the latest write.

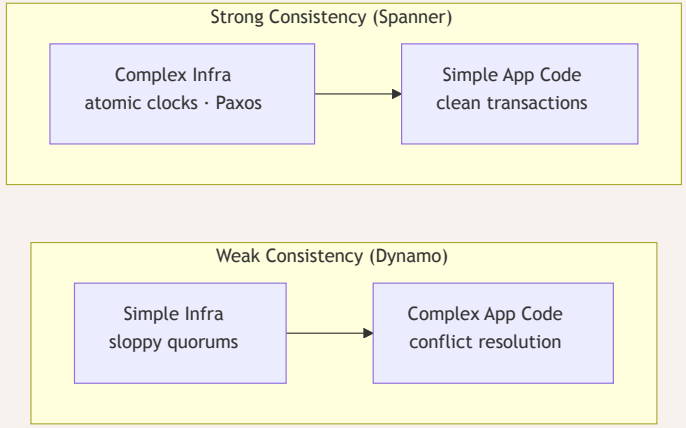
Three implementation strategies:

Strategy	Latency	Throughput	Example
Quorum reads	1 RTT to majority	Low	etcd (default)
Leader reads with lease	~0 (lease valid)	High	ZooKeeper
TrueTime	~7ms commit wait	Medium	Spanner

The fundamental cost: linearizability requires coordination. Coordination requires communication. Communication has latency.

At geo-distributed scale (100ms+ RTT between regions), linearizable reads add 50-100ms per read. Bounded-staleness reads add ~0ms but may be milliseconds behind.

Complexity doesn't disappear. It migrates.



The complexity is the same. The address is different.

Where the Field Is Heading

Everything so far was settled science by 2014.

Three new questions emerged.

THE FRONTIER

The Research Timeline

1979–2014: Defining the hierarchy

```
Lamport '79 → Herlihy & Wing '90 → Berenson '95 → CAP '00 → Dynamo '07 → Spanner '12 → PACELC '12 → HATs '14
```

2018–2025: Three new frontiers

Frontier	Key work	Question answered
Theory	CALM Theorem '19 → Keep CALM & CRDT On '22 → No-Op '25	When is coordination necessary?
Verification	Elle '21 → Jepsen finds MySQL / MariaDB / RDS bugs '23–'25	Does your database keep its promises?
Systems	Calvin → Anna → MixT → Hydro → Aurora DSQL → Durable Objects	Can we mix models safely per-operation?

The field shifted from **defining** consistency models to **automating the choice** between them.

THE FRONTIER

CALM Theorem: When Is Coordination Necessary?

Hellerstein & Alvaro (formalized 2019, CACM 2020):

> *A program has a consistent, coordination-free distributed implementation* ***if and only if*** *it is monotonic.*

Monotonic = only adds information, never retracts. Sets that grow, counters that increase, logs that append.

Non-monotonic = requires negation, aggregation, or thresholds. "At most 5 items," "no duplicates," "exactly one winner."

	CAP / PACELC	CALM
What it tells you	The trade-off exists	Whether the trade-off applies to **your** program
Granularity	System-wide	Per-computation

THE FRONTIER

Elle and the Verification Gap

The gap between what databases **claim** and what they **provide** is still real. Elle (Kingsbury & Alvaro, VLDB 2021) made it mechanically testable.

Elle: a black-box transactional consistency checker. Sound (no false positives), near-linear time, finds real bugs. Infers Adya-style dependency graphs from observed histories.

Recent Jepsen findings (2023–2025):

System	Year	Finding
MySQL 8.0.34	2023	Violates SERIALIZABLE on AWS RDS
MariaDB Galera	2025	Loses committed transactions during crashes
RDS PostgreSQL 17.4	2025	Long Fork anomalies; may only provide Parallel SI
NATS 2.12.1	2025	Data loss from lazy fsync (flush every 2 min)

THE FRONTIER

CRDTs Go Mainstream: Local-First Software

CRDTs moved from academic curiosity to production infrastructure. The **local-first** manifesto (Kleppmann et al., 2019) made "offline-first, sync later" a first-class architecture.

What changed:

- **Automerge 3.0**: Rust/WASM core. 10x memory reduction. A 200,000-edit document loads in under 4 seconds.
- **Fugue** (2023): first replicated list algorithm that prevents text interleaving — a previously unnoticed corruption bug in collaborative editing.
- **ConflictSync** (2025): 18x reduction in CRDT sync bandwidth via digest-driven reconciliation.
- **Blocklace** (2024): CRDTs extended to Byzantine fault tolerance — a CRDT that works even when nodes lie.

Industry adoption: Figma, Linear, and Notion all use CRDT-inspired approaches for real-time collaboration.

CRDTs are no longer a theory slide. They're the consistency model powering the tools you use every day.

The tools you use daily — Figma, Linear, Notion — already chose eventual consistency with CRDTs.

*The frontier is making that choice **automatic**.*

THE FRONTIER

Deterministic Ordering: A Third Path

Dynamo resolves conflicts **after** they happen. Spanner prevents conflicts **with clocks**. Calvin orders transactions **before** execution.

Calvin protocol (Thomson et al., SIGMOD 2012):

1. Batch incoming transactions into time windows
2. Replicate the batch via consensus (one round trip)
3. Every replica executes the same transactions in the same deterministic order

Result: strict serializability without atomic clocks, GPS, or distributed locking.

Trade-off: you must declare read/write sets upfront. Interactive transactions (read, think, write) are expensive because the system can't slot them into the global order until it knows what they'll touch.

FaunaDB built on Calvin and achieved Jepsen-verified strict serializability — proving the protocol works in production. The protocol continues to influence newer deterministic database designs.

THE FRONTIER

Mixed Consistency: Per-Operation Choice

The frontier: choose consistency **per operation**, with the compiler checking safety.

MixT (Milano & Myers, PLDI 2018):

Mix consistency levels within a single transaction. Compile-time information-flow analysis ensures weak data can't influence strong decisions. If your shopping cart is eventually consistent and your inventory is serializable, MixT prevents cart data from leaking into inventory checks.

Hydro Project (Berkeley, 2023–present):

A Rust framework that uses the CALM theorem at **compile time**. Lattice types in the Rust type system flag which operations need coordination. If your code is monotonic, Hydro compiles it without coordination. If it's not, you get a type error.

Instead of choosing one consistency model for your database, **your compiler tells you which operations need coordination and which don't.**

THE FRONTIER

Edge Computing Changes the Consistency Game

Edge platforms redefine the trade-off by moving compute to where data lives:

Cloudflare Durable Objects (2024 GA): Single-threaded actors with embedded SQLite. Code runs where data is stored — zero network hop for reads and writes. Strong consistency within each object. Eventual consistency between objects.

Aurora DSQL (AWS, 2024): Disaggregates query processing, transaction ordering, and storage into separate scalable components. Optimistic concurrency control with MVCC across regions.

The emerging pattern: partition data into small strongly-consistent units, accept weaker consistency between units.
This is the consistency analog of microservices — strong local, weak global.

2014: We defined all the consistency models.

2025: We're learning which ones you actually need — and building compilers that figure it out for you.

How to Choose

A decision framework, not a recommendation.

CHOOSING YOUR MODEL

Start From Your Invariants

Don’t ask: "which consistency model should I use?"

Ask: **"what invariants must my system never violate?"**

Financial invariant: "Account balance never goes negative" → Needs serializability (cross-object constraint)

Availability invariant: "Users can always add to cart" → Tolerates eventual consistency (conflicts are mergeable)

Ordering invariant: "Messages appear in send order" → Needs causal consistency (causality must be preserved)

Recency invariant: "Config changes take effect within 10 seconds" → Needs bounded staleness (timed convergence guarantee)

The invariant dictates the minimum model. Anything stronger is paying for guarantees you don't need.

CHOOSING YOUR MODEL

The Decision Variables

Four variables that drive the choice:

1. **Can conflicts be resolved automatically?** Yes → eventual consistency may suffice (CRDTs, last-writer-wins) No → you need coordination (serializability or above)

2. **Is cross-region latency acceptable?** Yes → linearizable / strict serializable is viable No → consider causal or session guarantees

3. **Is the data model append-only or mutable?** Append-only → weaker models are safer (events, logs) Mutable → stronger models prevent more anomalies (accounts, inventory)

4. **Who handles the complexity — infra or app?** Infra team is strong → stronger consistency, simpler app code App team is strong → weaker consistency, custom conflict handling

CHOOSING YOUR MODEL

Layer Relationships: Not All Models Are Alternatives

Before comparing, establish the relationship:

Alternative (pick one — same architectural layer):

- Linearizable vs. sequentially consistent (single-object guarantees)
- Serializable vs. snapshot isolation (transaction isolation levels)

Composable (use together — different layers):

- Linearizable objects + serializable transactions = strict serializable
- Eventual consistency + session guarantees (read-your-writes)

Orthogonal (solve different problems — coexist):

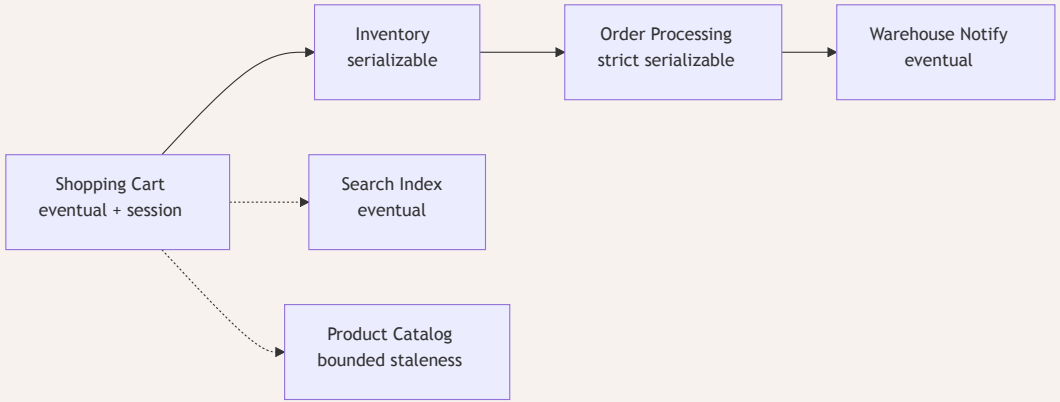
- System-level consistency vs. session-level guarantees

CHOOSING YOUR MODEL

What Most Production Systems Actually Use

The dirty secret: most production systems use **multiple consistency models** in the same application.

A typical e-commerce system:



One system, five different consistency choices. Each chosen by invariant, not by blanket policy.

Don't choose the model you want.
Choose the model your invariants require.

CHOOSING YOUR MODEL

Honest Trade-offs

What you gained:

- A precise vocabulary for what your database actually promises
- A hierarchy that relates every model to every other
- A framework for choosing: invariant → model → implementation

What you traded:

- "Eventually consistent" is no longer a comforting blanket term
- "We use a distributed database so it’s consistent" is no longer a valid claim
- You now have to ask your database vendor exactly which model they provide — and verify it (Jepsen exists for a reason)

SUMMARY

Three Things to Remember

1. **Two independent dimensions.** Linearizability (single-object, real-time) and serializability (multi-object, some order) are not the same axis. Strict serializability is both.

2. **Consistency has three layers.** System guarantees, session guarantees, and application invariants. They compose independently. Tune each one.

3. **Start from the invariant, not the model.** "What must never break?" → minimum consistency model → implementation. Most systems use multiple models in the same application.

RESOURCES

Reading List

Start here

- jepsen.io/consistency — the interactive hierarchy map (bookmark this)
- Berenson et al. "A Critique of ANSI SQL Isolation Levels." SIGMOD, 1995.
- Corbett et al. "Spanner: Google's Globally-Distributed Database." OSDI, 2012.
- Abadi. "Consistency Tradeoffs in Modern Distributed Database System Design." IEEE Computer, 2012.
- Hellerstein & Alvaro. "Keeping CALM: When Distributed Consistency is Easy." CACM, 2020.
- Kleppmann et al. "Local-First Software: You Own Your Data, in Spite of the Cloud." Onward!, 2019.

Go deeper

- Herlihy & Wing, TOPLAS 1990 (linearizability) · DeCandia et al., SOSP 2007 (Dynamo) · Bailis et al., VLDB 2014 (HATs) · Kingsbury & Alvaro, VLDB 2021 (Elle) · jepsen.io/analyses